

EJERCICIO. IA EN LOCAL. OLLAMA + OPEN WEB UI

Mayo-2026

Vamos a definir nuestra propia IA en LOCAL con OLLAMA + OPEN WEB UI

1. Instalar Ollama (El "Motor")

Ollama es el servicio que descarga y ejecuta los modelos de lenguaje (LLMs) en tu hardware.

1. **Descarga:** Ve a ollama.com y descarga la versión para tu sistema operativo (**Windows, macOS o Linux**).
2. **Instalación:** Ejecuta el instalador. En Windows y Mac, verás un icono de una pequeña "llama" en la barra de tareas cuando esté funcionando.
3. **Verificación:** Abre una terminal (CMD, PowerShell o Terminal de Mac) y escribe:

```
ollama -version
```

Si te responde con la versión, ¡estás listo!

2. Descargar un Modelo

Antes de abrir la interfaz web, necesitas un modelo que "piense". Vamos a bajar el más equilibrado actualmente (Llama 3.2 o Qwen 3, según tu preferencia).

En la misma terminal, escribe:

```
ollama run llama3.2
```

- Esto descargará el modelo (unos 2GB-5GB).
- Una vez termine, podrás chatear directamente en la terminal, pero vamos a pasar a lo bueno: la interfaz web.

3. Instalar Open WebUI (La "Interfaz")

La forma más limpia y recomendada de instalar Open WebUI es mediante **Docker**, ya que evita conflictos de dependencias en tu sistema.

1. **Instala Docker:** Si no lo tienes, descárgalo en docker.com. Asegúrate de que esté abierto.
2. **Ejecuta el contenedor:** Copia y pega este comando en tu terminal para conectar la interfaz con Ollama:

```
docker run -d -p 3000:8080 --add-host=host.docker.internal:host-gateway -v open-webui:/app/backend/data --name open-webui --restart always ghcr.io/open-webui/open-webui:main
```

¿Qué hace este comando?

- **-p 3000:8080:** Mapea la interfaz al puerto 3000 de tu navegador.
- **--add-host=host.docker.internal:host-gateway:** Crucial para que la interfaz "vea" a Ollama corriendo en tu máquina.
- **-v open-webui:/app/backend/data:** Guarda tus chats y configuraciones para que no se borren al apagar la PC.

4. Acceder y Configurar

1. Abre tu navegador y entra en: <http://localhost:3000>
2. **Registro:** Te pedirá crear una cuenta. **Ojo:** Todo es local, esos datos no salen de tu ordenador; es solo para gestionar tus propios chats. El primer usuario creado será el Administrador.
3. **Selecciona el modelo:** En la parte superior verás un desplegable. Selecciona el modelo que descargaste antes (llama3.2).

Consejos "Pro" para 2026

- **Acelera la carga:** Si tienes una tarjeta gráfica NVIDIA, asegúrate de tener instalados los drivers con soporte **CUDA**. Ollama los detectará automáticamente y usará la GPU en lugar del procesador, haciendo que la IA responda 10 veces más rápido.

- **Explora modelos:** Puedes ir a la biblioteca de Ollama y probar otros modelos como mistral, deepseek-coder (para programar) o phi3 (muy ligero para PCs modestas).
- **Actualizaciones:** Para actualizar Open WebUI a la última versión, solo tienes que detener el contenedor en Docker y volver a ejecutar el comando del Paso 3; Docker bajará la imagen más reciente automáticamente.

Con Ollama y Open WebUI en 2026, el ecosistema local ha evolucionado muchísimo. Ya no es solo texto; ahora tienes una estación de trabajo de IA completa. Aquí te detallo qué puedes hacer exactamente:

1. Multimodalidad (Visión de Imágenes)

Ollama soporta modelos multimodales que te permiten "subir" imágenes para que la IA las analice.

- **Qué puedes hacer:** Adjuntar fotos, capturas de pantalla o diagramas y preguntar: *"¿Qué dice este gráfico?"*, *"Explicame este código de la captura"* o *"Describe esta imagen"*.
- **Modelos recomendados:** * llama3.2 (en sus versiones Vision).
 - llama-v1.6 o el nuevo gemma3 (con soporte nativo de visión).
 - moondream (muy ligero, ideal si no tienes mucha potencia).

2. Generación de Imágenes (Integración)

Aunque Ollama por sí mismo no genera imágenes (es un motor de lenguaje), **Open WebUI** tiene una integración genial.

- **Cómo funciona:** Puedes conectar Open WebUI con un motor de generación como **Stable Diffusion** o **ComfyUI** (que corran en tu PC) o incluso con **DALL-E 3** vía API.
- **El resultado:** En el chat verás un botón para generar imágenes basadas en tu conversación.

3. Documentos y RAG (Tu propia base de conocimientos)

Esta es quizá la función más potente para productividad y consultoría:

- **Análisis de archivos:** Puedes arrastrar PDFs, archivos Word, Excel o CSV directamente al chat.
- **RAG Local:** Open WebUI indexa el documento y puedes hacerle preguntas sobre su contenido. Por ejemplo: *"Basado en este informe de ventas, ¿cuál fue el producto*

estrella en marzo?". Todo se procesa en tu máquina, por lo que tus datos están seguros.

4. ¿Y el Video?

A día de hoy (mayo 2026), el soporte de video funciona de dos formas:

- **Análisis de frames:** Algunos modelos multimodales avanzados pueden analizar una secuencia de imágenes (frames) de un video corto para describirlo.
- **Llamadas de Voz y Video:** Open WebUI incluye una función de "Hands-free" (manos libres) donde puedes hablarle a la IA y ella te responde con voz (usando motores locales como Whisper para escucharte y Piper o OpenAI para hablar).

Resumen de capacidades en tu entorno local:

Función	Estado en 2026	Requisito
Texto/Chat	Perfecto	Cualquier modelo (llama3.2, qwen3)
Análisis Imágenes	Nativo	Modelos tipo llava o llama-vision
Generación Imágenes	Integrado	Necesitas Stable Diffusion o API externa
Análisis Documentos	Excelente	Función RAG nativa de Open WebUI
Voz (STT/TTS)	Muy fluido	Configurar Whisper (local) en la UI
Generación Video	Experimental	Generalmente requiere herramientas externas

Tip Extra: Como mencionaste que trabajas con **pymes y automatización**, la función de **Documentos (RAG)** combinada con modelos de razonamiento como deepseek-r1 te permitiría crear un consultor de IA que analice manuales internos de una empresa sin que ni un solo dato salga a internet.

Si lo quieres hacer con PODMAN en vez de DOCKER

¡Claro que sí! De hecho, usar **Podman** es una alternativa excelente si buscas una gestión de contenedores que no dependa de un demonio central y que sea, por defecto, *rootless* (más segura).

Como Podman es mayoritariamente compatible con la sintaxis de Docker, el proceso es casi idéntico, pero hay un par de detalles clave con la red y los permisos que debes tener en cuenta.

Aquí tienes los pasos para hacerlo con Podman:

1. Preparar Ollama

Asegúrate de que Ollama esté corriendo en tu máquina "host" (fuera de Podman).

- Si lo instalaste como aplicación nativa en Windows o macOS, simplemente ábrelo.
 - **Importante:** Por defecto, Ollama solo escucha peticiones desde localhost. Para que el contenedor de Podman pueda hablar con él, debes configurar la variable de entorno OLLAMA_HOST a 0.0.0.0.
-

2. Ejecutar Open WebUI con Podman

En tu terminal, usa el siguiente comando. He ajustado la comunicación para que sea compatible con el modo en que Podman gestiona el tráfico hacia el host:

Bash

```
podman run -d \  
-p 3000:8080 \  
--name open-webui \  
--add-host=host.containers.internal:host-gateway \  
-e OLLAMA_BASE_URL=http://host.containers.internal:11434 \  
-v open-webui-data:/app/backend/data \  
--restart always \  
ghcr.io/open-webui/open-webui:main
```

¿Qué ha cambiado aquí?

- **host.containers.internal:** Es el equivalente en Podman al host.docker.internal de Docker. Permite que el contenedor salga a buscar a Ollama en tu máquina real.
 - **-e OLLAMA_BASE_URL:** Le decimos explícitamente a la interfaz dónde está el "cerebro" (Ollama).
 - **Volumen:** Podman creará un volumen llamado open-webui-data automáticamente para persistir tus datos.
-

3. Consideración de Seguridad (SELinux)

Si estás en una distribución tipo **Fedora, RHEL o CentOS**, SELinux podría bloquear el acceso al volumen. Si notas que el contenedor no arranca o da errores de permisos, añade :Z al final del volumen: -v open-webui-data:/app/backend/data:Z

4. Verificación

Una vez ejecutado, espera unos segundos y comprueba que el contenedor esté arriba:

Bash

```
podman ps
```

Luego, entra en <http://localhost:3000>. Si al intentar elegir un modelo no aparece nada, asegúrate de que el firewall de tu sistema no esté bloqueando el puerto 11434, que es por donde Ollama recibe las órdenes.

Ventaja adicional de Podman

Si usas **Podman Desktop**, puedes gestionar este contenedor desde una interfaz gráfica muy intuitiva, similar a Docker Desktop pero sin las restricciones de licencia para empresas que este último tiene.

Si quiero montar ollama en KUBUNTU

Para montar este entorno en **Kubuntu 24.04 (Noble Numbat)**, tienes la ventaja de que es un sistema muy estable y excelente para manejar contenedores. Al ser una distribución basada en Ubuntu, el proceso es muy fluido.

Aquí tienes la ruta óptima para Kubuntu:

1. Instalar Ollama de forma nativa

En Linux, lo mejor es instalar Ollama como un servicio del sistema para que aproveche directamente el kernel y los drivers de tu GPU.

Abre la terminal (Konsole) y ejecuta:

Bash

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **Verificación:** Una vez termine, el servicio debería estar activo. Pruébalo con: `systemctl status ollama`

2. Preparar Podman en Kubuntu

Kubuntu 24.04 tiene Podman en sus repositorios oficiales. Instalémoslo junto con las herramientas necesarias para que los contenedores se comuniquen bien:

Bash

```
sudo apt update
```

```
sudo apt install -p podman podman-docker uidmap
```

3. Configurar Ollama para conexiones externas

Por defecto, Ollama en Linux escucha solo en 127.0.0.1. Para que el contenedor de Podman pueda hablar con él, debemos abrirlo un poco:

1. Edita el servicio de Ollama:

Bash

```
sudo systemctl edit ollama.service
```

2. Se abrirá un archivo vacío (o con comentarios). Pega esto debajo de [Service]:

Ini, TOML

[Service]

```
Environment="OLLAMA_HOST=0.0.0.0"
```

```
Environment="OLLAMA_ORIGINS=*"
```

3. Guarda (Ctrl+O, Enter) y cierra (Ctrl+X).
4. Reinicia el servicio para aplicar los cambios:

Bash

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart ollama
```

4. Lanzar Open WebUI con Podman

Ahora ejecutamos el contenedor. Usaremos el flag `--network=host` para simplificar la vida en Kubuntu, ya que permite que el contenedor comparta la red con tu sistema y vea a Ollama instantáneamente en `localhost:11434`.

Bash

```
podman run -d \  
--network=host \  
-v open-webui:/app/backend/data \  
-e OLLAMA_BASE_URL=http://127.0.0.1:11434 \  
--name open-webui \  
--restart always \  
ghcr.io/open-webui/open-webui:main
```

5. Soporte para GPU (NVIDIA) en Kubuntu

Si tienes una tarjeta NVIDIA, querrás que la IA vuele. En Kubuntu 24.04 necesitas el "NVIDIA Container Toolkit" para que Podman pueda usar la tarjeta:

1. **Instala los drivers** (si no lo has hecho): `sudo ubuntu-drivers install` y reinicia.
2. **Instala el toolkit:**

Bash

```
sudo apt install -y nvidia-container-toolkit
```

3. **Configura Podman:**

Bash


```
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml
```

4. **Ejecuta el contenedor con GPU:** Añade el flag `--device nvidia.com/gpu=all` al comando de `podman run` del paso anterior.

Bonus: ¿Prefieres Interfaz Gráfica?

Como estás en KDE (Kubuntu), puedes instalar **Podman Desktop**. Es una aplicación nativa que te permitirá ver los logs, detener o actualizar Open WebUI sin tocar la terminal. Está disponible como **Flatpak** en la tienda **Discover**.

¡Ya puedes entrar en `http://localhost:8080` (o el puerto que hayas definido) y empezar a descargar modelos! ¿Qué procesador y cuánta RAM tiene esa máquina? Ayuda mucho para saber qué modelos te irán más finos.